

WHAM-HBRIDGE-PFMG474

Firmware Build, Flash, and Verification

Standard Operating Procedure

September 2026

Rev	Date	Author	Description
A	2026-08-31	–	Initial issue: build, bootstrap flash, serial reflash. No power-stage procedures yet (Section 2).
B	2026-09-04	–	Added Section 10: uploading a PFM table and firing it – this firmware now drives real HRTIM PWM output (Section 2 updated accordingly). Serial baud raised 9600 → 115200 throughout (Section 9 and elsewhere).
C	2026-09-29	–	Build is now python/wham_build.py (images published to build/); added the NRST pull-up requirement (Section 4) and the in-application network update (Section 8); *IDN? now reports the git commit.

1 Purpose

This document specifies the procedure for building, flashing, and verifying WHAM-HBRIDGE-PFMG474 firmware, and for confirming a board is running the expected firmware version before it is put into service or handed off.

2 Scope

Covers firmware build and flashing (Sections 5–9) and, as of Rev B, uploading a PWM shot profile and firing it (Section 10) – this firmware **now drives real HRTIM PWM output** on six pins (three complementary phase pairs) the moment FIRE is sent. **This firmware still has no ARM**

precondition, no state machine, and no fault interlock – FIRE takes effect immediately and unconditionally whenever it is sent, by design (see docs/command_reference.md). Treat any gate driver or power stage wired to these pins as live for the duration of Section 10, per your lab’s normal HV/gate-driver procedure, exactly as if driven by any other PWM source – this SOP provides no closed-loop protection behind it. This document still does not cover power-stage bring-up, calibration, or safety-interlock procedures beyond that caution; a companion procedure remains needed once a state machine/interlock exists in firmware.

3 Prerequisites

3.1 Equipment

Item	Notes
WHAM-HBRIDGE-PFMG474 board	Powered per its own hardware documentation (not covered here).
ST-Link (or equivalent SWD debug probe)	Required for the bootstrap flash (Section 6) and for recovery if a board stops responding over serial entirely.
USB-to-serial adapter	Wired to USART2: adapter TX → PA3 (USART2_RX), adapter RX → PA2 (USART2_TX), GND ↔ GND.

(TX/RX wiring above is crossed: adapter TX goes to the MCU’s RX pin and vice versa, as usual for a point-to-point serial link.)

3.2 Software

Tool	Notes
STM32CubeIDE stm32flash	Project build and the bootstrap SWD flash. AN3155 USART bootloader client. macOS: brew install stm32flash.
Python 3 + pyserial	Runs python/wham_serial_flash.py. pip install pyserial.

CAUTION: This firmware has not yet been through an ESD-controlled production process. Use standard ESD precautions (grounding strap, static-safe surface) when handling the bare board.

4 Hardware Requirement: NRST Pull-Up

Every board must have a firm pull-up on the NRST line (a resistor to +3V₃; 5.1 kΩ was verified on the WHAM-XREX boards, which use the same

PCB). The STM32G474 has no pull-down on NRST, only a weak internal pull-up that is disabled during every internal reset, and something else on the board holds the net partway down (measured 1.6 V, between the input-low and input-high thresholds). Without the resistor the chip can stay in reset after a firmware swap or a cold power-on, silent on the serial link, until an ST-Link is attached. Before investigating a “silent after swap/power-on” report, measure NRST: healthy is near 3.3 V. Use a resistor, not a hard tie to 3.3 V (it defeats the chip’s internal-reset holder). With the pull-up fitted, attach the ST-Link with SWDIO, SWCLK, GND and 3V3 only and leave its NRST wire off.

5 Procedure: Build the Firmware

1. From the project root, run:

```
python3 python/wham_build.py
```

This regenerates the git-version header, checks that the IDE build lists match the source tree and that the source layout rules hold, runs the host unit tests, builds the **Debug** configuration, generates the .bin (the IDE build does not, so a bare make leaves a stale one), and publishes WHAM-HBRIDGE-PFMG474.{elf,bin} to build/.

2. Confirm the build produced no warnings and every test passed.
3. In STM32CubeIDE, F5 (Refresh) and a normal Build also work for development, but they do not produce the .bin or the build/ copies; use wham_build.py for anything you intend to flash.

6 Procedure: Bootstrap Flash (SWD)

Required the *first* time a board receives firmware with the serial-bootloader feature (BOOT command), since that feature is what the serial reflash path in Section 7 depends on – a board cannot use it to receive the very first firmware that adds it. Also use this procedure to recover a board that no longer responds to BOOT over serial at all.

1. Connect the ST-Link to the board’s SWD header and to the host computer.
2. In STM32CubeIDE, press **Run** (or **Debug**) to build and flash via the ST-Link, as normal for this IDE. From the command line, with SWDIO, SWCLK, GND and 3V3 connected and the ST-Link’s **NRST wire left disconnected** (see Section 4):

```

openocd -f interface/stlink.cfg -f target/stm32g4x.cfg
\
-c "adapter speed 400; reset_config none" -c init -c
halt \
-c "program build/WHAM-HBRIDGE-PFMG474.bin
0x08000000 verify" \
-c "reset run" -c exit

```

3. Proceed to Section 9 to confirm the board is running the expected firmware.

7 Procedure: Serial Reflash

Use this for all firmware updates *after* the bootstrap flash in Section 6 has been done once. This is the remote-friendly path: no physical BOOTo/NRST/SWD access is required, only the USART2 serial link.

1. With the board powered and running its current firmware, and the USB-serial adapter connected, run:

```

python3 python/wham_serial_flash.py \
--port /dev/cu.usbserial-XXXXX

```

substituting the actual serial device path. (On macOS, a single USB-serial adapter shows up as both a `cu.*` and `tty.*` device node; the script sees this as two candidates and will not auto-detect – `--port` must be passed explicitly. Prefer the `cu.*` node.)

2. The script will: send B00T and wait for the acknowledgement; run `stm32flash` to write and verify `build/WHAM-HBRIDGE-PFMG474.bin`; then start the new firmware automatically (no manual reset needed – see Section 7.1).
3. Confirm the script reports [done] with firmware written and verified before proceeding.
4. Proceed to Section 9.

7.1 Note on automatic restart after flashing

Early testing of this procedure found that the newly-flashed firmware could come up unresponsive over serial until the board was manually power-cycled, because the STM32 ROM bootloader's *Go* command (used to start the new firmware without a physical reset) does not perform a true chip reset, and left the processor's vector table pointer aimed at the bootloader's own vector table rather than the application's. This has been fixed in firmware (`boot_jump.c`) – a manual reset should not be necessary.

If a board flashed under this SOP does not respond to the verification step below without a manual reset, that is a regression and should be reported, not treated as expected behavior.

8 Procedure: In-Application Update (Network or Serial)

New in Rev C. Updates the firmware over the ordinary command link (115200 8N1), so it works through the ethernet serial bridge, which cannot carry the ROM bootloader's 8E1. The new image is written into the flash bank that is *not* running; the running image is untouched until the final swap and stays in the other bank, so `--rollback` can boot it again. The first image containing this feature must be flashed with the ST-Link (Section 6).

1. Build (Section 5). Make sure no table is playing.
2. Run, with the link up (for the network case, through `net_broker.py` so others can watch):

```
python3 python/fw_update.py \  
--url socket://localhost:5001 --name fw_update
```

The tool identifies the board, erases the inactive bank, streams the image with a CRC check, verifies it, swaps banks and waits for the !BOOT banner of the new image.

3. Confirm it ends with `[ok] now running <version> <commit>`.
4. To go back to the previous image: `fw_update.py --rollback`.

CAUTION: The image must contain this firmware's own product name (WHAM-HBRIDGE-PFMG474); another product's image is refused. The swap resets the chip – see Section 4 for the NRST requirement.

9 Procedure: Verification

1. Open a serial connection to the board at **115200 8N1** (raised from 9600 as of Rev B – see `docs/changelog.txt`).
2. Send `*IDN?` followed by a line ending (`\r`, `\n`, or both).
3. Confirm a reply of the form:

```
OK <board name> <hardware rev> <firmware version>  
<git commit>
```

e.g. OK WHAM-HBRIDGE-PFMG474 REVA v0.7 008dc35d. A -dirty suffix on the commit means the firmware was built from an uncommitted working tree.

4. Confirm the reported firmware version matches the version that was just built/flushed.

Acceptance criterion for this SOP: Step 3 above returns a well-formed reply with the expected version, on the first query, with no manual reset performed since the flash in Section 5 or 7.

10 Procedure: Upload and Fire a PFM Table

New in Rev B. Uploads a complete PWM shot profile (a sequence of period/duty entries, one per switching cycle) and plays it back on demand. Full protocol detail: docs/command_reference.md's TABLE:* and FIRE entries.

CAUTION: FIRE drives real HRTIM PWM output on six pins (three complementary phase pairs, ~100ns hardware dead time between each pair) the instant it is sent. There is no ARM step, no state machine, and no fault interlock in this firmware – FIRE always takes effect immediately, whether the board was idle or a previous shot was still running. Treat any gate driver or power stage wired to these pins as live before proceeding, per your lab's normal HV/gate-driver procedure.

1. Complete Sections 5–9 first – a board must already be running current, verified firmware.
2. Table *construction* (what shot profile to build) lives entirely on the host, not in firmware – see pfm.h's own header note for why. python/pfm_table_upload.py is the reference builder: edit its PROFILE constant (1–5; see the script's own header comment for what each one is) before running:

```
python3 python/pfm_table_upload.py \  
--port /dev/cu.usbserial-XXXXX
```

3. Confirm the script reports [done] table uploaded and confirmed with the expected entry count (TABLE:END -> OK <count>).
4. Optionally confirm independently at any time with TABLE? over a serial terminal (e.g. python/scpi.py) – replies OK <count>, matching the upload.

5. With the CAUTION above acknowledged and anything downstream of these six pins confirmed safe to drive, send FIRE. A prompt OK confirms the shot started; output stops on its own, at a coherent period boundary, once the table is exhausted – there is no separate STOP command and no completion notification.
6. Sending FIRE against an empty table (nothing uploaded yet) is rejected with ERR 5 and drives nothing – this is the one built-in safeguard, not a substitute for the interlock discussed above.

Acceptance criterion for this SOP: TABLE:END reports the exact entry count the host script intended to send, and FIRE returns OK with no ERR. Confirming the actual output waveform (dead time, phase spacing, duty accuracy) is a bench/instrumentation task outside this SOP's scope – see docs/test_protocol.md's T4 group if that level of verification is needed.

11 Troubleshooting

Symptom	Cause / action
stm32flash reports Failed to init device, timeout. before any write occurs	Not a firmware issue – this probe doesn't depend on what's running on the chip. Check board power, serial wiring (TX/RX not crossed, common GND), and that no other program has the port open.
B00T gets no reply, but power/wiring check out	The firmware currently on the board may predate the B00T command (run the bootstrap flash, Section 6) or may be a board affected by the Go-jump issue in Section 7.1 – power-cycle once, then retry.
Verification (Section 9) fails after a serial reflash	Power-cycle the board once and repeat verification. If it then succeeds, this is the regression described in Section 7.1 – report it. If it still fails, treat as a failed flash and repeat Section 7 from the start.

12 References

- AGENTS.md – project orientation and current functionality.
- docs/command_reference.md – full serial command protocol, including TABLE:* and FIRE (Section 10).
- docs/serial_reflash_guide.md – detailed reflash mechanism, including the Go-jump bug referenced in Section 7.1.

- docs/test_protocol.md – bench-level verification, including waveform checks beyond this SOP’s acceptance criteria.
- python/pfm_table_upload.py – reference table builder used in Section 10; see its own header comment for what each PROFILE value builds.
- docs/changelog.txt – dated development history.